Classification Visualized

Machine Learning Assignment 2 | Five classification models on one public dataset

a. Problem statement

Predict whether a breast mass is benign or malignant from measurements of cell nuclei. Compare Logistic Regression, Decision Tree, K-Nearest Neighbors (KNN), Gaussian Naive Bayes and Support Vector Machine (SVM) on the same held-out data using Accuracy, ROC AUC, Precision, Recall, F1 and Matthews Correlation Coefficient (MCC). Provide an interactive Streamlit app that evaluates saved models on uploaded labeled test CSVs without retraining.

b. Dataset description

Dataset: Breast Cancer Wisconsin (Diagnostic), UCI Machine Learning Repository.

Source: UCI dataset page. **Citation:** Wolberg, W., Mangasarian, O., Street, N., & Street, W. (1993). Breast Cancer Wisconsin (Diagnostic) [Dataset]. DOI: 10.24432/C5DW2B. Dataset license: CC BY 4.0.

Property	Description
Task	Binary classification
Instances	569 (minimum required: 500)
Input features	30 numeric measurements (minimum required: 12)
Target	diagnosis: 0 = benign, 1 = malignant
Full-dataset class counts	Benign: 357; malignant: 212
Missing values	None in the original dataset
Training partition	455 rows: 285 benign, 170 malignant
Held-out test partition	114 rows: 72 benign, 42 malignant

The features come from digitized fine-needle aspirate images. Ten measurements describe radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension. Each has a mean, standard error and worst value (mean of the three largest values), giving 30 inputs. IDs are excluded.

The source is loaded offline using `sklearn.datasets.load_breast_cancer`, which bundles this UCI dataset. The training script saves a complete local copy in `data/breast_cancer_wisconsin.csv`. Project headers use `mean_`, `error_` and `worst_` prefixes; spaces in measurement names become underscores. The native scikit-learn targets are inverted so malignant is positive. These changes and dataset attribution are documented in `data/README.md`.

c. Github Repository Link

https://github.com/2026aj05482/classification-visualized

Streamlit deployment status: Prepared and tested locally. A live Community Cloud deployment has not yet been verified. Deployment settings and instructions appear below.

d. Models used

Every model uses all 30 input features and the same stratified 80/20 train/test split with `random_state=42`. Hyperparameters were fixed before evaluating the test set; no test-set tuning was performed. There is no cross-validation or claim of an optimized model. The test partition remains excluded from fitting.

StandardScaler is fitted only on the training partition within each saved pipeline for Logistic Regression, KNN, Gaussian Naive Bayes and SVM. The Decision Tree uses unscaled inputs. No imputation is necessary because the source dataset has no missing values; incomplete uploaded rows are rejected. Each `.joblib` file includes the fitted classifier and its preprocessing.

ML Model Name	Configuration
Logistic Regression	L2 regularization, C=1.0, max_iter=2000
Decision Tree	Gini criterion, max_depth=5, min_samples_leaf=2
KNN	5 neighbors, uniform weights, Euclidean distance
Naive Bayes	GaussianNB, var_smoothing=1e-9
Support Vector Machine	RBF kernel, C=1.0, gamma=scale

Evaluation metrics comparison

Scores below are computed by loading the saved pipelines and evaluating the exact 114-row `test_data.csv`. Values are on a 0-1 scale, except MCC which ranges from -1 to 1. Values are rounded to four decimal places; ten decimal places are retained in `reports/metrics.csv`.

ML Model Name	Accuracy	AUC	Precision	Recall	F1	MCC
Logistic Regression	0.9649	0.9960	0.9750	0.9286	0.9512	0.9245
Decision Tree	0.9123	0.9444	0.9444	0.8095	0.8718	0.8113
KNN	0.9561	0.9823	0.9744	0.9048	0.9383	0.9058
Naive Bayes	0.9211	0.9891	0.9231	0.8571	0.8889	0.8292
Support Vector Machine	0.9737	0.9947	1.0000	0.9286	0.9630	0.9442

Accuracy measures the proportion of correct predictions. Precision = TP/(TP+FP), recall = TP/(TP+FN), and F1 is their harmonic mean, all with malignant (1) as the positive class. MCC measures correlation between the actual and predicted labels using all four confusion-matrix cells. AUC measures ranking across thresholds using continuous scores: malignant probabilities for four models, and the decision function for SVM. Predictions use each classifier's standard `predict()` rule; SVM scores are not presented as probabilities.

Observations about model performance

ML Model Name	Observation about model performance
Logistic Regression	Correct on 110/114 rows. Detects 39/42 malignant cases with 1 false positive and 3 false negatives. Highest AUC (0.9960); strong ranking with a simple linear model.
Decision Tree	Correct on 104/114 rows. Detects 34/42 malignant cases with 2 false positives and 8 false negatives. Lowest recall, F1, AUC and MCC in this comparison. Axis-aligned partitions may fit these relationships less effectively; this split does not establish the cause.
KNN	Correct on 109/114 rows. Detects 38/42 malignant cases with 1 false positive and 4 false negatives. High precision (0.9744), but recall is below Logistic Regression and SVM. Scaling makes the distance calculation less dominated by large-magnitude features.
Naive Bayes	Correct on 105/114 rows. Detects 36/42 malignant cases with 3 false positives and 6 false negatives. AUC (0.9891) is strong despite lower threshold-based performance. The independence assumption may be restrictive for related nuclear measurements.
Support Vector Machine	Correct on 111/114 rows. Detects 39/42 malignant cases with no false positives and 3 false negatives. Highest accuracy, precision, F1 and MCC; recall ties Logistic Regression. AUC is slightly below Logistic Regression.
Overall, Winner for your dataset?	Support Vector Machine for the strongest combined threshold-based results: accuracy 0.9737, F1 0.9630 and MCC 0.9442. Logistic Regression wins on AUC. SVM leads accuracy by only one test case; this is a descriptive result for this holdout, not proof of universal superiority.

The conclusions apply to this 114-row split. A broader comparison would use repeated or nested cross-validation and an independent external test dataset. Saved detailed classification reports and confusion matrices are in `reports/evaluation.json`.

Repository contents

```
classification-visualized/
|-- app.py                          # Streamlit inference dashboard
|-- ml_utils.py                     # CSV validation and shared metrics
|-- train_models.py                 # Reproducible offline training
|-- requirements.txt                # Pinned app and training dependencies
|-- requirements-dev.txt            # Additional PDF-generation dependencies
|-- README.md                       # Assignment report and instructions
|-- test_data.csv                   # 114 held-out rows with target
|-- model/                          # Five fitted .joblib pipelines + metadata
|-- data/                           # Complete dataset + attribution
|-- reports/                        # Metrics CSV + evaluation JSON
|-- scripts/build_submission.py     # Render this README to PDF
|-- output/pdf/submission.pdf       # PDF containing the README content
|-- tests/test_project.py           # Artifact, validation and app checks
|-- .streamlit/config.toml          # Theme and 5 MB upload limit
```

`model/metadata.json` records feature order, class encoding, split indices, package versions and SHA-256 checksums. The five model files are `logistic_regression.joblib`, `decision_tree.joblib`, `knn.joblib`, `naive_bayes.joblib` and `support_vector_machine.joblib`.

Run locally

Use Python 3.13 (the tested version). From the repository directory:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
python -m streamlit run app.py
```

On macOS/Linux, activate with `source .venv/bin/activate`. Saved artifacts are included, so running the app needs no training and no dataset download. The app reads its files relative to `app.py`, independent of the current working directory.

To reproduce the trained artifacts and evaluation files explicitly:

```
python train_models.py
python -m unittest discover -s tests -v
```

The integration checks verify split isolation, train-only scaling, agreement between saved artifacts and reported metrics, all five model selections, uploaded CSV evaluation, invalid CSV handling and undefined AUC for single-class uploads. Fitting is blocked during the inference tests to detect accidental retraining. To regenerate the submission PDF after updating this README:

```
python -m pip install -r requirements-dev.txt
python scripts/build_submission.py
```

Streamlit app features and CSV usage

1. Start with the bundled held-out dataset to see all five models immediately.

2. Download `test_data.csv` from the sidebar to obtain the exact CSV schema.

3. Select Upload test CSV and upload a labeled test file. It must contain all 30 feature columns plus `diagnosis`. Feature order may vary. The target accepts 0/1 or B/M, where 1/M is malignant. Remove extra ID/index columns.

4. Inspect the six-metric comparison table for all five models on the same uploaded rows. Select a model from the dropdown for metric cards, confusion matrix, classification report and row predictions. ROC curves compare all five models. Comparison and prediction CSV downloads are available.

Uploads are limited to 5 MB and 10,000 rows. Numeric features must be finite and complete. Duplicate headers, missing features, extra columns, nonnumeric values and invalid labels produce readable validation messages. The app accepts only CSV uploads and loads only the repository's saved models. It never fits a model or scaler on uploaded data.

If an uploaded file has only one target class, ROC AUC is mathematically undefined and is shown as N/A; ROC curves are omitted. Undefined precision/recall/F1 divisions and degenerate MCC are reported as 0, matching the documented evaluation convention. Use a test file with both classes for a complete six-metric comparison.

Deploy on Streamlit Community Cloud

1. Push all project files, including `model/` and `test_data.csv`, to the GitHub repository above.

2. Sign in at Streamlit Community Cloud using GitHub, then choose Create app and indicate that you already have an app.

3. Repository: `2026aj05482/classification-visualized`; branch: `main`; entrypoint: `app.py`.

4. Under Advanced settings, select Python 3.13 to match the tested environment, then deploy. No secrets are required.

5. Confirm that the six metrics, all five comparison rows, model dropdown, confusion matrix and upload workflow work on the live URL. Record that URL in this README and regenerate the PDF before final submission.

The root `requirements.txt` includes every direct app dependency and pins the numerical libraries used to serialize the models. Do not upgrade scikit-learn independently of the artifacts; retrain and verify if changing versions. PDF-only dependencies are separated so Community Cloud does not need them. Official guidance: Deploy your app on Community Cloud.